ĐẠI HỌC BÁCH KHOA HÀ NỘI

Học phần IT5440 – Nguyên lý và kỹ thuật phân tích chương trình (kỳ 2026.1)

Báo cáo chạy thử (pilot)

Dựng ngữ cảnh bằng đồ thị gọi hàm và hiệu ứng vị trí

Đề tài 5-KH – Chương 13: Mô hình ngôn ngữ lớn cho hiểu mã nguồn và phân tích chương trình

Nhóm: KHMT 2 (lớp Khoa học máy tính CH2026B)
Căn cứ: Đề cương bản nháp 1 – mục II.9.1, bước 4
Ngày: 11/11/2026

Tình trạng tài liệu. Đây là báo cáo kết quả chạy thử với một mô hình, cỡ mẫu nhỏ và mã tổng hợp, không phải kết quả chính thức của đề tài. Các số trong ngoặc vuông (ví dụ [2]) và mã RQ, R, K theo đánh số của đề cương bản nháp 1.

Mục lục

1. Tóm tắt.....3
2. Mục đích và thiết lập.....3
3. Hiệu chỉnh độ khó.....4
4. Kết quả.....4
5. Phân tích lỗi và kiểm tra tính hợp lệ.....5
6. Ý nghĩa, hạn chế và bước tiếp theo.....6
7. Cấu hình phần cứng và tốc độ.....6
8. Tái lập.....8

1. Tóm tắt

Nhóm đã chạy thử theo bước 4 của mục II.9.1 trong đề cương, với mô hình mở qwen2.5-coder:7b chạy cục bộ qua Ollama, trên bộ dữ liệu sinh tự động kiểu SemTrace-Dep và bốn điều kiện ngữ cảnh C0–C3. Kết quả chính của lần chạy 2 (50 mục, lời nhắc thực tế khoảng 3,7k token):

- **Hàm gây nhiễu làm giảm mạnh độ chính xác:** từ 68% (C0, chỉ mã cần thiết) xuống còn 12–32% (C1, ngữ cảnh dài thô).
- **Bao đóng đồ thị gọi hàm (C3) khôi phục hoàn toàn độ chính xác:** 74%, ngang C0 và cao hơn rõ rệt so với C1 ở vị trí giữa (McNemar chính xác, $p < 0{,}0001$). Đây là bằng chứng ban đầu cho RQ2.
- **Chỉ sắp xếp lại (C2) không giúp được:** 26%, không khác C1 ở vị trí giữa ($p = 0{,}65$).
- **Không thấy hiệu ứng "lost in the middle" như [2]:** vị trí đầu và giữa bằng nhau (khoảng 30%), vị trí cuối thấp nhất (12%, khác có ý nghĩa so với đầu và giữa, $p \approx 0{,}04$–$0{,}05$). Đây là kết quả của RQ1 ở mức pilot, cần kiểm tra thêm với mô hình khác.

2. Mục đích và thiết lập

Mục đích. Kiểm tra bộ sinh dữ liệu, bộ dựng ngữ cảnh và quy trình chấm điểm trước khi chạy chính, ước lượng chi phí và thời gian thực tế, và có kết quả sơ bộ cho RQ1 và RQ2.

Bộ dữ liệu (SemTrace-Dep rút gọn). Mỗi mục có một hàm đích gọi 3 hàm phụ (một chuỗi gọi độ sâu 2 và một hàm phụ gọi trực tiếp), trả về một danh sách hai số nguyên. Mỗi hàm phụ gồm 2 phép cộng/trừ với hằng số ba chữ số và một nhánh if, để hàm không rút gọn được thành "đầu vào + hằng số" (khắc phục điểm yếu đã nêu ở Phụ lục A của đề cương). Hàm gây nhiễu gồm hai loại với tỉ lệ ngang nhau: giống cấu trúc (cùng mẫu, khác hằng số, không được gọi) và hàm tiện ích tổng quát do nhóm tự viết (không vướng giấy phép). Đáp án lấy bằng thực thi và được kiểm chéo bằng lần thực thi thứ hai.

Bảng 1. Các điều kiện ngữ cảnh (theo Bảng 12 của đề cương)

Mã	Định nghĩa
C0	Chỉ các hàm cần thiết, callee trước caller (cận trên)
C1	Toàn bộ mã gồm hàm gây nhiễu; các hàm cần thiết rải rác quanh vị trí tương đối $p \in \{0; 0{,}5; 1\}$
C2	Cùng nội dung và độ dài như C1, nhưng mã cần thiết gom thành một khối ở cuối, trước câu hỏi
C3	Bao đóng đồ thị gọi hàm tĩnh, tính bằng thư viện ast của Python trên chính ngữ cảnh C1 (không dùng đáp án của bộ sinh)

C4 (truy xuất nhúng) chưa cài đặt trong pilot này.

Mô hình và lời nhắc. qwen2.5-coder:7b (họ mô hình có trong [2]), chạy cục bộ qua Ollama trên MacBook 16 GB RAM; nhiệt độ $T = 0$. Câu hỏi đặt trước và sau mã như [2]. **Khác với [2]:** mô hình được yêu cầu truy vết từng bước rồi ghi đáp án ở dòng cuối (biến thể P1 ở mục II.8.5), vì khi trả lời trực tiếp mô hình đạt 0% (mục 3). Đáp án là danh sách cuối cùng xuất hiện trong câu trả lời; đúng khi khớp chính xác.

Lý do chọn mô hình. (1) Mục II.8.5 của đề cương yêu cầu ít nhất một mô hình thuộc danh sách của [2] làm mốc kiểm tra tái lập, ví dụ một biến thể Qwen 2.5 Coder; (2) mô hình miễn phí, chạy cục bộ, không cần ngân sách API (ngân sách chưa chốt, mục II.15); (3) bản 7B lượng tử hóa 4 bit chiếm khoảng 5 GB, vừa máy 16 GB. Kích thước cụ thể mà [2] dùng chưa được đối chiếu. Danh sách mô hình cho lần chạy chính vẫn do nhóm quyết định.

Thống kê. Khoảng tin cậy Wilson 95% cho từng điều kiện; kiểm định McNemar chính xác (hai phía) cho các cặp điều kiện trên cùng tập mục. Câu trả lời không phân tích được tính là sai. Chưa hiệu chỉnh Holm vì đây là pilot.

3. Hiệu chỉnh độ khó

Mục tiêu là C0 đạt khoảng 50–90%: nếu C0 gần 0% thì không điều kiện nào khác biệt được, và không đo được hiệu ứng vị trí.

Bảng 2. Các lần hiệu chỉnh độ khó

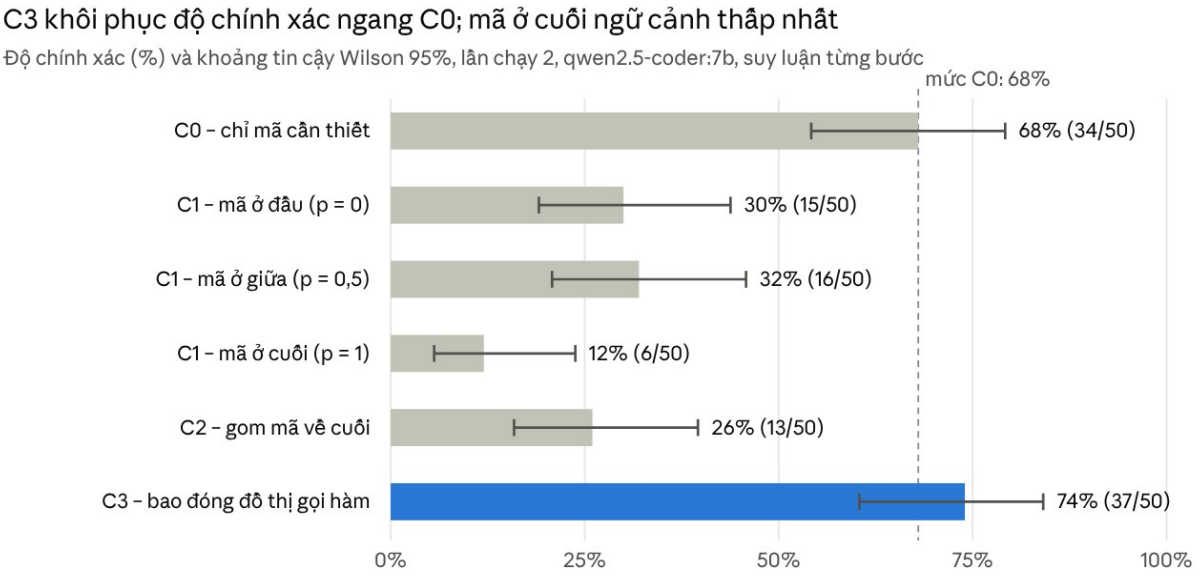
Lần thử	Thiết lập	C0	Ghi chú
v1	3 phép tính mỗi hàm, có nhân ×2/×3, trả lời trực tiếp, n = 20	0%	Mọi điều kiện đều 0%; mô hình đoán (ví dụ [100, 100])
v2 mặc định	2 phép cộng/trừ, trả lời trực tiếp, n = 10	0%	
v2 dễ nhất	Độ sâu 1, 2 hàm phụ, 1 phép tính, trả lời trực tiếp, n = 10	0%	
v2 dễ nhất + từng bước	Như trên, cho phép suy luận từng bước, n = 10	80%	C3: 90%
v2 mặc định + từng bước	Độ sâu 2, 3 hàm phụ, 2 phép tính, n = 10	70%	Thiết lập được chọn

Nhận xét. Ở cỡ 7B, mô hình không tính được kết quả của ngay cả một hàm phụ một bước nếu phải trả lời trực tiếp; câu trả lời chỉ khoảng 11 token và là phỏng đoán. Vì vậy các lần chạy chính dùng suy luận từng bước. Điều này khác với thiết lập của [2] và cần nêu rõ khi so sánh.

4. Kết quả

4.1. Lần chạy 2 (chính): 50 mục

Thiết lập: qwen2.5-coder:7b, suy luận từng bước, giới hạn đầu ra 2.048 token, seed 2, 300 truy vấn trong 3 giờ 11 phút. Tham số --tokens 2000 thực tế tạo lời nhắc khoảng 3,7k token (tối đa 3.885) vì bộ đếm token của script ước lượng thấp; cửa sổ ngữ cảnh 8.192 token nên không có lời nhắc nào bị cắt.



Hình 1. Độ chính xác theo điều kiện, lần chạy 2 (n = 50 mỗi điều kiện; thanh lỗi: khoảng tin cậy Wilson 95%).

So sánh ghép cặp với C1 ở vị trí giữa (McNemar chính xác, hai phía). Cột "Tốt hơn" là số mục điều kiện kia đúng còn C1 giữa sai; "Kém hơn" là ngược lại.

Bảng 3. So sánh ghép cặp, lần chạy 2

So sánh	Tốt hơn	Kém hơn	p
C3 so với C1 giữa	24	3	< 0,0001
C0 so với C1 giữa	20	2	0,0001
C1 đầu so với C1 giữa	11	12	1,0
C1 cuối so với C1 giữa	5	15	0,041
C2 so với C1 giữa	8	11	0,65
C1 đầu so với C1 cuối	13	4	0,049

C3 cải thiện rõ rệt so với ngữ cảnh thô, và C2 không cải thiện. Vị trí cuối kém hơn cả đầu lẫn giữa, nhưng p = 0,04–0,05 là sát ngưỡng và sẽ không còn ý nghĩa nếu hiệu chỉnh Holm cho nhiều so sánh.

4.2. Lần chạy 1: 20 mục, ngữ cảnh dài hơn

Thiết lập: như lần 2 nhưng --tokens 4000 (lời nhắc thực tế dài hơn con số này), giới hạn đầu ra 1.024 token, seed 1, 20 mục.

Bảng 4. Độ chính xác, lần chạy 1 (n = 20)

Điều kiện	Độ chính xác
C0	65%
C1 đầu	25%
C1 giữa	5%
C1 cuối	5%
C2	15%
C3	75%

C3 so với C1 giữa: 14 tốt hơn, 0 kém hơn (p = 0,0001). C1 ở cả ba vị trí đều gần 0%, nên lần chạy này không phân biệt được các vị trí (hiệu ứng sàn). Đây là lý do lần chạy 2 dùng ngữ cảnh ngắn hơn và nhiều mục hơn.

5. Phân tích lỗi và kiểm tra tính hợp lệ

Câu trả lời bị cắt. Ở lần chạy 2, 32 trên 300 câu trả lời không có đáp án cuối, và 31 trong số đó chạm giới hạn 2.048 token. Phân bố theo điều kiện: C2 10, C1 đầu 7, C1 cuối 6, C1 giữa 6, C3 2, C0 1. Nhiều câu không chỉ dài mà bị lặp vòng: mô hình lặp lại cùng một lời gọi với giá trị tăng dần (ví dụ x tăng thêm 81 sau mỗi dòng), hoặc đi theo lời gọi của hàm gây nhiễu. Vì vậy nâng giới hạn từ 1.024 lên 2.048 token không giải quyết được. Các câu này được tính là sai. Lặp vòng gần như chỉ xảy ra khi có hàm gây nhiễu, nên đây là một dạng lỗi đáng báo cáo trong phần phân tích sai.

Lỗi của bộ sinh: trùng tên hàm. Ở mục 39 (p = 0,5), một hàm gây nhiễu nhận ngẫu nhiên cùng tên với hàm đích (fn_810b), nên ngữ cảnh định nghĩa hàm này hai lần. Đáp án lưu sẵn vì thế sai cho ngữ cảnh đó (ảnh hưởng tối đa 1 mục ở C1 giữa và 1 mục ở C3). Đây cũng là lý do C3 khớp chính xác 149/150 thay vì 150/150. Cách sửa: hàm sinh tên phải loại các tên đã dùng. Cần sửa trước lần chạy tiếp theo.

Vị trí cuối có bị cắt ngữ cảnh không? Không: lời nhắc dài nhất khoảng 3,9k token, cửa sổ ngữ cảnh 8.192 token. Vì vậy kết quả "cuối kém nhất" không phải do cắt cụt, tuy nhiên nguyên nhân thực sự chưa được kiểm

tra. Một giả thuyết cần thử: khi suy luận từng bước, mô hình đọc từ trên xuống và bị cuốn vào các hàm gây nhiễu giống cấu trúc trước khi tới mã cần thiết.

C3 đúng gần như tuyệt đối trên mã tổng hợp. Đồ thị gọi hàm tĩnh theo tên tìm đúng tập hàm cần thiết ở 149/150 ngữ cảnh, đúng như rủi ro R4 trong đề cương đã dự báo. Kết quả của C3 vì thế là mức trần khi phân tích tĩnh hoàn hảo, chưa phải kết quả trên mã thực.

6. Ý nghĩa, hạn chế và bước tiếp theo

Liên hệ với câu hỏi nghiên cứu.

- **RQ1 (hiệu ứng vị trí):** pilot không tái hiện dạng chữ U của [2]. Với mô hình này và cách hỏi này, hàm gây nhiễu gây hại ở mọi vị trí, và vị trí cuối có xu hướng kém nhất. Kết quả âm này vẫn có giá trị (đề cương đã thiết kế để kết quả âm có thể công bố, rủi ro R2), nhưng cần thêm mô hình trước khi kết luận.
- **RQ2 (dựng ngữ cảnh):** bằng chứng ban đầu rõ nhất. Chọn mã theo đồ thị gọi hàm (C3) khôi phục độ chính xác về mức C0 ở cả hai lần chạy, còn chỉ sắp xếp lại (C2) thì không. Điều này phù hợp với rủi ro R3 theo chiều ngược lại: cải thiện đến từ việc loại bỏ hàm gây nhiễu, không phải từ thứ tự.
- **RQ3 (kiểm chứng bằng thực thi):** chưa thử trong pilot.

Hạn chế. Một mô hình 7B; một seed mỗi lần chạy; mã tổng hợp, nơi phân tích tĩnh gần như hoàn hảo; suy luận từng bước thay vì trả lời trực tiếp như [2]; độ dài ngữ cảnh thực tế khoảng 3,7k token, ngắn hơn nhiều so với 16k của [2]; 32/300 câu bị cắt; một lỗi trùng tên trong bộ sinh; chưa hiệu chỉnh đa so sánh. C4 (truy xuất nhúng) chưa có.

Bước tiếp theo (đề xuất, theo thứ tự ưu tiên).

1. Sửa lỗi trùng tên trong bộ sinh và bộ đếm token (dùng số token thật do Ollama trả về).
2. Lặp lại lần chạy 2 với một mô hình thứ hai (ví dụ một mô hình Llama hoặc một mô hình thương mại qua API với tập con) và một seed khác, để xem "cuối kém nhất" có lặp lại không.
3. Cài đặt C4a (truy xuất nhúng) để có đường cơ sở cho K5.
4. Thêm biến thể gọi gián tiếp (hàm truyền như tham số, tra cứu qua từ điển) để C3 không còn đúng tuyệt đối (rủi ro R4).
5. Chốt cách trả lời (từng bước hay trực tiếp) và ghi vào cấu hình đóng băng trước khi chạy chính.

Chi phí thực tế. Chạy cục bộ nên không tốn phí API; khoảng 230–280 giây mỗi mục (6 truy vấn) trên MacBook 16 GB. Một lần 50 mục mất khoảng 3–4 giờ, nên chạy qua đêm.

7. Cấu hình phần cứng và tốc độ

7.1. Máy chạy pilot (đo thực tế)

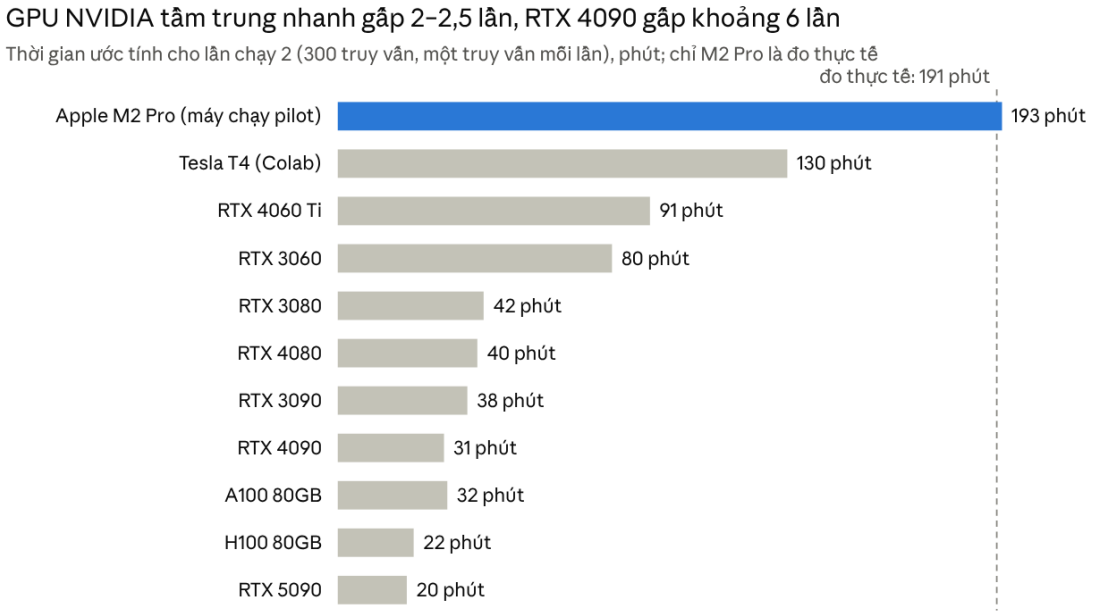
Bảng 5. Cấu hình máy và phần mềm

Thành phần	Giá trị
Máy	MacBook Pro (Mac14,9), chip Apple M2 Pro
CPU	10 nhân (6 hiệu năng + 4 tiết kiệm điện)
GPU	16 nhân, Metal
Bộ nhớ	16 GB bộ nhớ hợp nhất (CPU và GPU dùng chung), băng thông tối đa 200 GB/s
Phần mềm	macOS (bản dựng 26A434), Ollama 0.40.16 (dùng llama.cpp trên Metal)
Mô hình	qwen2.5-coder:7b, 7,6 tỉ tham số, lượng tử hóa 4 bit (Q4_K_M), 4,7 GB; khi chạy chiếm 5,0 GB, 100% trên GPU
Tốc độ đo	Đọc lời nhắc khoảng 277 token/s; sinh khoảng 29 token/s (hai truy vấn khoảng 3,7k token, không dùng bộ nhớ đệm)

Ở lần chạy 2, mỗi truy vấn trung bình có 2.593 token vào và 858 token ra, mất 38,2 giây. Tổng 777.894 token vào ÷ 277 token/s cộng 257.316 token ra ÷ 29,4 token/s cho 11.561 giây, khớp với thời gian thực tế 11.470 giây (sai 1%). Khoảng 75% thời gian là sinh token, nên tốc độ sinh (phụ thuộc chủ yếu vào băng thông bộ nhớ) quyết định thời gian chạy.

7.2. So sánh với GPU NVIDIA (ước tính, chưa đo)

Cách ước tính. Lấy số liệu công khai của llama.cpp cho cùng một phép đo (Llama 2 7B, lượng tử hóa Q4_0, đọc 512 token và sinh 128 token, không flash attention) trên từng thiết bị [discussion #4167 cho Apple Silicon, #15013 cho NVIDIA CUDA]. Trên bảng đó M2 Pro 16 nhân GPU đạt 294 token/s khi đọc và 37,9 token/s khi sinh; nhóm đo được 277 và 29,4 (mô hình lớn hơn và ngữ cảnh dài hơn), tức hệ số 0,94 và 0,78. Áp hai hệ số này cho từng GPU rồi tính lại thời gian cho khối lượng công việc của lần chạy 2. Cách này dự đoán 193 phút cho chính M2 Pro, so với 191 phút đo thực tế.



Hình 2. Thời gian ước tính cho lần chạy 2 trên từng thiết bị (một truy vấn mỗi lần).

Bảng 6. Số liệu llama.cpp công khai (Llama 2 7B Q4_0, không flash attention)

Thiết bị	Bộ nhớ	Đọc pp512 (token/s)	Sinh tg128 (token/s)
Apple M2 Pro 16 GPU	16 GB hợp nhất	294	38
Tesla T4	16 GB GDDR6	1.219	46
RTX 4060 Ti	8 GB GDDR6	3.395	64
RTX 3060	12 GB GDDR6	2.138	76
RTX 3080	10 GB GDDR6X	5.014	140
RTX 4080	16 GB GDDR6X	8.032	142
RTX 3090	24 GB GDDR6X	5.175	158
RTX 4090	24 GB GDDR6X	11.993	186
A100	80 GB HBM2e	4.850	191
H100	80 GB HBM3	9.918	268
RTX 5090	32 GB GDDR7	14.073	290

Nhận xét.

- GPU NVIDIA đọc lời nhắc nhanh hơn M2 Pro 4–48 lần, nhưng vì 75% thời gian là sinh token nên lợi ích thực tế nhỏ hơn: RTX 3060 nhanh khoảng 2,4 lần, RTX 3090/4090 khoảng 5–6 lần, T4 trên Colab miễn phí chỉ khoảng 1,5 lần.
- Đây là ước tính cho một truy vấn mỗi lần như pilot. Chạy song song nhiều truy vấn (ví dụ bằng vLLM trên GPU NVIDIA) sẽ nhanh hơn nhiều; trên Mac 16 GB thì không đủ bộ nhớ để làm việc này.
- Phần cứng chỉ đổi thời gian, không đổi độ chính xác: cùng mô hình, cùng lượng tử hóa và T = 0 cho kết quả gần như giống nhau (sai khác nhỏ về số học dấu phẩy động có thể đổi vài câu trả lời).
- **Hệ quả cho kế hoạch:** lần chạy chính theo đề cương (ít nhất 3 mô hình, thêm C4 và độ dài 16k) sẽ mất nhiều ngày trên Mac này. Nếu nhóm có GPU của phòng thí nghiệm (RTX 3090 trở lên) thì nên dùng cho lần chạy chính.

Nguồn: github.com/ggml-org/llama.cpp/discussions/4167 (Apple Silicon) và github.com/ggml-org/llama.cpp/discussions/15013 (NVIDIA CUDA), truy cập 11/11/2026.

8. Tái lập

Mã nguồn và kết quả nằm trong thư mục pilot/ (chỉ dùng thư viện chuẩn của Python 3 và Ollama).

```
ollama pull qwen2.5-coder:7b
python3 pilot.py --dry-run --n 5
# Lần chạy 1
python3 pilot.py --model qwen2.5-coder:7b --n 20 --cot --tokens 4000 --seed 1
# Lần chạy 2
python3 pilot.py --model qwen2.5-coder:7b --n 50 --cot --tokens 2000 \
    --max-out 2048 --seed 2
```

Bảng 7. Các tệp trong thư mục pilot/

Tệp	Nội dung
pilot.py	Bộ sinh dữ liệu, bộ dựng ngữ cảnh C0–C3, gọi mô hình, chấm điểm (bản v3; pilot_v1.py và pilot_v2.py là các bản trước)
results/qwen2.5-coder_7b_20261111_012743.jsonl	Nhật ký lần chạy 1 (120 truy vấn)
results/qwen2.5-coder_7b_20261111_071717.jsonl	Nhật ký lần chạy 2 (300 truy vấn)
run_v2_cot.log, run_v3_2k.log	Bảng kết quả và thời gian chạy
calib_*.log	Các lần hiệu chỉnh ở mục 3

Mỗi dòng trong tệp .jsonl ghi một truy vấn: mục, điều kiện, vị trí, đáp án đúng, câu trả lời thô, đáp án đã phân tích, đúng/sai và số token. Với cùng seed, bộ sinh tạo lại đúng các mục cũ.